

# Contents

|                                                                |          |
|----------------------------------------------------------------|----------|
| <b>guide 01 training verifiers to solve math word problems</b> | <b>2</b> |
| 0. 先给新手的结论                                                     | 2        |
| 1. 当时的历史背景                                                     | 3        |
| 2. 论文自己的结构地图                                                   | 3        |
| 3. 论文主张拆成问题、方法、证据                                              | 3        |
| 问题                                                             | 3        |
| 方法                                                             | 4        |
| 证据                                                             | 4        |
| 4. GSM8K 数据集设计                                                 | 4        |
| High Quality                                                   | 4        |
| High Diversity                                                 | 5        |
| Moderate Difficulty                                            | 5        |
| Natural Language Solutions                                     | 5        |
| 5. Baseline: finetuning generator                              | 5        |
| 为什么必须生成完整解法                                                    | 6        |
| 6. test@1 和 test@100: generator 不能只看单样本                        | 6        |
| 7. Verification pipeline                                       | 6        |
| 8. Token-level verifier: 不只看最后一刻                               | 7        |
| Solution-level verifier                                        | 7        |
| Token-level verifier                                           | 7        |
| 9. Joint objective: verifier 也要懂语言                             | 8        |
| 10. Generator 和 verifier 谁更大                                   | 8        |
| 11. Test-time compute: 样本越多不总是越好                               | 9        |
| Top-ranked voting                                              | 9        |
| 12. Dropout: 正则化不是小细节                                          | 9        |
| 13. Calculator annotations                                     | 9        |
| 14. 数学公式汇总                                                     | 10       |
| Generator finetuning                                           | 10       |
| Sampling candidates                                            | 10       |
| Labeling verifier data                                         | 10       |
| Token-level verifier loss                                      | 10       |
| Joint verifier objective                                       | 10       |
| Test-time selection                                            | 10       |
| Top-ranked voting                                              | 11       |
| 15. 代码样例: sample-and-rank                                      | 11       |
| 16. 代码样例: final answer label                                   | 11       |
| 17. 代码样例: top-K voting                                         | 11       |
| 18. 和本仓库代码的对应关系                                                | 12       |
| 19. 论文的局限和后来工作的方向                                              | 12       |
| Outcome label 不等于过程正确                                          | 12       |
| Verifier 会被搜索攻击                                                | 12       |
| 数据集难度后来被追上                                                     | 13       |
| Calculator 和 tool-use 改变问题定义                                   | 13       |
| 20. 对今天的意义                                                     | 13       |
| 21. AI agent 正确学习这篇论文的方法                                       | 14       |
| 22. 读完必须能闭卷回答                                                  | 14       |
| 23. 一页复盘                                                       | 14       |

# guide\_01\_training\_verifiers\_to\_solve\_math\_word\_problems

原论文: Training Verifiers to Solve Math Word Problems

本地原文 PDF: [learning/reasoning-eval/paper/01\\_gsm8k.pdf](#)

作者: Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, et al., OpenAI

年份: 2021

读法定位: 这篇论文同时做了两件大事。第一, 它发布了 GSM8K, 一个后来成为 LLM 数学推理标准入口的数据集。第二, 它系统展示了 verifier/sample-and-rank 思路: 让 generator 生成很多候选解, 再训练 verifier 从候选解里挑最可能正确的。这是后来 best-of-N、reward model reranking、process reward model、test-time compute scaling 的早期清晰版本。

## 0. 先给新手的结论

这篇论文的核心不是“模型会做小学数学了”, 而是:

生成一个正确解很难。

判断一个候选解是否正确, 通常比从零生成正确解容易。

所以可以把 reasoning 拆成 generation + verification。

整体流程:

```
math word problem
  |
  v
generator samples many solutions
  |
  v
verifier scores each solution
  |
  v
select highest-scored solution
  |
  v
extract final answer and evaluate
```

论文最需要记住的数字:

- GSM8K 有 8.5K 道高质量小学数学文字题。
- 切分为 7.5K train 和 1K test。
- 每题通常需要 2 到 8 步。
- 解法是自然语言推理过程, 最后给最终答案。
- 作者估计 breaking errors 或歧义少于 2%, 二次抽查 disagreement 约 1.7%。
- generator 训练 2 个 epoch 后采样 100 个 completion 给 verifier 训练。
- test time 默认也采样 100 个 completion, 由 verifier 排序。
- 6B verification 在 full dataset 上略好于 finetuned 175B, 论文称相当于约 30x model size boost。
- 只让 6B 模型直接输出 final answer, 不写中间步骤, 性能从 20.6% 掉到 5.2%。

一句话压缩:

GSM8K 让“多步数学推理”变成可重复评测的问题; verifier 让“多采样后选择”变成可训练的 test-time compute 方法。

## 1. 当时的历史背景

2021 年，大模型已经在很多 NLP 任务上很强。GPT-3 证明了 scale 和 in-context learning 的威力。但数学推理暴露出一个硬伤：多步推理对单步错误极其敏感。

自回归模型生成解题过程时，一旦中间某一步错了，后面 token 往往会沿着错误继续走。模型没有天然机制回头修正。数学题正好放大这个问题：

```
question interpretation error
-> wrong equation
-> wrong arithmetic
-> wrong final answer
```

更糟的是，语言流畅不等于推理正确。模型可以生成看起来很像标准答案的解释，但里面某一步算错或概念用错。只看文本自然度没有用，必须检查最终答案，最好还能检查过程。

当时已有一些 math word problem 数据集，但常见问题是：

- 太小，难以测试现代大模型。
- 只有方程或最终答案，没有自然语言推理过程。
- 题目模板化严重，训练/测试差异不大。
- 质量控制弱，答案或解法错误较多。
- 难度要么太低，要么像 MATH 那样对当时模型过难，进展难测。

GSM8K 的定位就是这个“甜点区”：对大模型足够难，但对人类中学生可解；题目多样，解法自然语言化，适合观察模型的 informal reasoning。

## 2. 论文自己的结构地图

建议按这个顺序读原文：

1. Abstract 和 Introduction: 抓住 GSM8K 和 verifier 的双重贡献。
2. Section 2 Dataset: 看数据集为什么强调 high quality、high diversity、moderate difficulty、natural language solutions。
3. Section 4 Methods: 先读 finetuning baseline，再读 verification pipeline。
4. Figure 3: 理解为什么 generator 只训练 2 epoch，而不是训练越久越好。
5. Figure 4: 记住 verifier 训练的三步: finetune generator, sample 100 completions, train verifier。
6. Figure 5: 看 verification 相对 finetuning 的数据 scaling 证据。
7. Figure 6: 看 token-level verifier、joint objective、generator/verifier size 的 ablation。
8. Figure 7: 看 test-time compute 为什么不是样本越多越好。
9. Figure 8: 看 dropout 为什么是重要 regularizer。
10. Appendix C/E/F: 看 calculator annotation、verifier head、token-level value visualization。

## 3. 论文主张拆成问题、方法、证据

### 问题

大型语言模型会写流畅文本，但在多步数学文字题上容易因为单个中间错误导致整题失败。单纯扩大模型或增加 finetuning 数据，按论文的外推看并不划算。

## 方法

作者提出:

1. 构造 GSM8K: 高质量、多样化、自然语言解法的 grade school math 数据集。
2. 训练 generator: 让模型学会产生完整自然语言解法。
3. 训练 verifier: 给模型候选解打分, 判断候选解最终是否正确。
4. test time 多采样: 从 generator 采样多个候选解, 用 verifier 选最优。

## 证据

实验显示:

- full solution 比 direct final answer 重要。
- generator 训练太久会损失多样性, test@100 变差。
- verification 在足够数据下明显优于 finetuning baseline。
- token-level verifier 比 solution-level verifier 更抗过拟合。
- joint LM + verifier objective 好于只做 verifier objective。
- test-time samples 增加到一定程度有益, 但过多会产生骗过 verifier 的 adversarial solutions。
- dropout 对 finetuning 和 verification 都有显著帮助。

## 4. GSM8K 数据集设计

GSM8K 的全名可以理解为 Grade School Math 8K。它有 8.5K 道题, 其中 7.5K train, 1K test。

每个样本大致是:

question:

A natural language word problem.

solution:

Step-by-step natural language reasoning.

Optional calculator annotations.

Final answer line.

典型解法格式:

She eats  $3 + 4 = \langle\langle 3+4=7 \rangle\rangle 7$  eggs.

She sells  $16 - 7 = \langle\langle 16-7=9 \rangle\rangle 9$  eggs.

She makes  $9 * 2 = \langle\langle 9*2=18 \rangle\rangle 18$  dollars.

#### 18

这里  $\langle\langle 3+4=7 \rangle\rangle$  是 calculator annotation。它不是人类承包商原始写出来的, 而是后续用规则和 fine-tuned LM 自动加上去的。训练时它只是普通 token; 测试时如果模型生成了格式正确的 annotation, 系统会用 calculator 覆盖接下来的 value token。

## High Quality

作者没有从网上大规模 scrape, 而是找人写题。之后让不同 worker 重新解题, 检查最终答案是否一致。有 disagreement 的题会修复或丢弃。二次抽查发现约 1.7% 仍有 disagreement, 作者估计 breaking errors 或歧义少于 2%。

这个设计很重要: 数学 benchmark 如果 gold answer 错了, 会直接污染所有后续模型比较。

## High Diversity

作者主动避免模板题。例如很多旧数据集会出现:

Alice has X apples, gives Y away...

Bob has X candies, gives Y away...

这种题的语言变化小, 模型可以靠模板匹配。GSM8K 要求题目场景和语言尽量多样, 还用 pairwise similarity 给承包商反馈, 减少重复模板。

## Moderate Difficulty

题目主要需要 elementary arithmetic, 加减乘除和早期代数。论文说一个 bright middle school student 应该能解所有题。大多数题不需要显式设变量。

这不是因为作者想做简单题, 而是为了让数据集处在可测区间:

- 太容易: 模型很快饱和, 看不出方法差异。
- 太难: 所有模型接近 0, 也看不出进步。
- GSM8K: 2021 年大模型很难, 但不是不可攻克。

## Natural Language Solutions

作者选择自然语言解法, 而不是只收方程。原因:

- 可以训练模型形成可读的 internal monologue。
- 能观察模型是否真的推理。
- 未来更容易和人类解释、verifier、过程监督连接。

这也是 GSM8K 后来成为 chain-of-thought 标准演示数据集的重要原因。

## 5. Baseline: finetuning generator

finetuning baseline 很直接:

```
pretrained GPT-3 family model
    |
    v
finetune on GSM8K question + solution
    |
    v
test: sample one low-temperature solution
    |
    v
check final answer
```

训练目标是标准 language modeling cross entropy:

$$L_{lm} = - \sum_t \log p_{\theta}(token_t | token_{<t})$$

实验使用 GPT-3 family, 主要看 6B 和 175B, 也有 3B、12B。Figure 2 显示模型越大、数据越多, finetuning performance 越高, 但增长不够快。作者做了一个粗略外推: 如果只靠 full GSM8K 上 finetuning, 要达到 80% solve rate, 可能需要约  $10^{16}$  参数级别; 沿数据方向可能也需要至少额外两个数量级的数据。

这个外推不一定是现代结论, 但它在论文里的作用很明确: 只靠“更大 generator”不是理想路径, 需要更 favorable scaling 的方法。

## 为什么必须生成完整解法

论文有一个非常关键的 ablation:

如果 6B 模型被 finetune 成直接输出 final answer, 不写中间步骤, 性能从 20.6% 掉到 5.2%。  
这说明中间自然语言推理不是装饰。它给模型提供了分解问题、保存中间数值、逐步计算的空间。

对今天的学习启发:

direct answer:

`x -> y`

reasoning trace:

`x -> step_1 -> step_2 -> ... -> y`

trace 增加 token 成本, 但降低了一次性从题目跳到答案的难度。

## 6. test@1 和 test@100: generator 不能只看单样本

论文 Figure 3 非常重要。作者训练 6B 模型 100 个 epoch, 分别看:

- test@1: 每题一次低温采样是否答对。
- test@100: 每题 100 次较高温采样中是否至少有一次答对。

结果:

- test@1 大体随训练继续提升。
- test@100 很快达到峰值, 然后随着训练更久而下降。

原因是 overconfidence 和 coverage collapse。训练太久后, 模型对训练分布更自信, 但生成的候选解多样性变差。对于单样本 accuracy, 过拟合不一定马上显现; 对于多样性搜索, 候选空间覆盖变差会很伤。

这解释了为什么 verifier pipeline 里 generator 只 finetune 2 epoch:

good verifier needs diverse candidate solutions

diverse candidate solutions need a generator with high test@N coverage

too much finetuning improves test@1 but hurts coverage

这是一个很深的设计理由。verifier 不可能从一堆全错且相似的候选里挑出正确答案。

## 7. Verification pipeline

论文 Figure 4 的 pipeline:

1. Finetune generator for 2 epochs on GSM8K train.
2. For each train problem:
  - sample 100 completions from generator
  - extract final answer
  - label completion correct if final answer matches gold
3. Train verifier for 1 epoch on these labeled completions.
4. At test time:

```
sample 100 completions
score each completion with verifier
return highest-scored completion
```

一个候选样本是:

```
x = question
y = generated full solution
a = extracted final answer from y
gold = gold answer
z = 1 if a == gold else 0
```

verifier 学的是:

```
v_phi(x, y) -> probability that y is correct
```

注意标签只看最终答案是否正确。它不是人工逐步标注。这样便宜, 但有两个问题:

- false positive: 推理过程错了, 但最后答案碰巧对。
- false negative: 解法合理但 answer extraction 或题目歧义导致被标错。

论文 Appendix F 的 visualization 就展示了 verifier 也会错。

## 8. Token-level verifier: 不只看最后一刻

verifier 可以有两种:

### Solution-level verifier

只在完整解法结束后输出一个分数:

```
v(x, y_1:m) -> score
```

### Token-level verifier

在解法的每个 token 后都输出一个 value:

```
v_t = v(x, y_1:t)
```

训练标签仍然是整条 completion 的正确/错误  $z$ , 但每个 solution token 位置都预测  $z$ 。

张量级图示:

```
input tokens:
[Q1, Q2, ..., Qn, S1, S2, ..., Sm]
```

```
language modeling objective:
predict next solution token
question tokens are masked out
```

```
verifier objective:
for each solution position t:
predict z in {0, 1}
```

outputs:

```
logits[t, vocab]
verifier_value[t]
```

论文的 verifier 架构很巧:

- verifier 本质仍是语言模型。
- 在 final unembedding logits 里保留一个 special token。
- 用一个 bias 和一个 gain 去 shift/scale 这个 special token 的 logit。
- 这个 special token logit 就作为 verifier scalar prediction。
- 其他 token 仍可服务 language modeling objective。

Figure 6a 显示 token-level verifier 最终优于 solution-level verifier, 而且更不容易过拟合。直觉是: token-level value 迫使模型沿着推理过程判断, 而不是只记最终答案模式。

## 9. Joint objective: verifier 也要懂语言

作者训练 verifier 时不只做 correctness prediction, 还保留 LM objective。

简化目标:

$$L = L_{lm} + L_{verifier}$$

$L_{verifier}$  = mean over solution token positions:  
 $(v_t - z)^2$

论文使用 MSE 作为 verifier loss。训练数据是 verifier data 和 language data 的 equal mix。因为每个原始 training problem 会采样 100 个 completion, equal mix 相当于把原始 language data upsample 100 倍。

Figure 6b 显示 joint objective 严格好于 verification-only。直觉是: verifier 要判断解法, 必须理解 generator 生成的语言分布。只学二分类可能很快过拟合 final-answer shortcut; 保留 LM objective 能作为辅助正则。

## 10. Generator 和 verifier 谁更大

Figure 6c 分别改变 generator size 和 verifier size。

结论:

```
large generator + small verifier
> small generator + large verifier
```

这很重要。原因是 verifier 只能在候选解集合里挑。如果 generator 根本没有生成正确候选, verifier 再强也没用。

可以把 test-time 成功拆成两步:

```
P(success)
= P(generator produces at least one correct candidate)
  * P(verifier selects a correct candidate | one exists)
```

generator 控制第一项, verifier 控制第二项。第一项为 0 时, 第二项没有意义。

论文还提出一个有趣解释: verifier 可能并不在做完整严格证明, 而是在学习相对粗粒度的 heuristics 来区分某个 generator 产生的好/坏解。这也是为什么小 verifier 仍能有效。



## 11. Test-time compute: 样本越多不总是越好

verification 的一个吸引力是可以在 test time 花更多算力:

```
sample N solutions
rank by verifier
choose best
```

Figure 7a 显示 6B verifier 随 completion 数量增加, 性能一开始提升, 到约 400 个 completion 继续受益; 超过这个点后开始下降。

为什么会下降? 因为搜索空间越大, 越可能找到骗过 verifier 的 adversarial solutions。也就是说 verifier 不是完美 judge。你给它看足够多候选, 总会有一些错误解法恰好被它高估。

论文默认使用 100 completions, 是成本和收益的折中。

### Top-ranked voting

论文还试了 top-ranked voting:

```
sample many solutions
rank by verifier
take top K ranked solutions
vote by final answer
return majority answer
```

Figure 7b 的结论:

- 如果只有 100 samples, top 3-5 vote 比较合适。
- 如果有 3200 samples, 可以让 top 30 左右参与 vote。

这已经很接近后来常见的 consensus、self-consistency、best-of-N reranking 思路。

## 12. Dropout: 正则化不是小细节

Figure 8 显示 residual dropout 对 finetuning 和 verification 都很重要。作者使用 20% dropout。

关键细节:

- GPT-3 预训练时没有 dropout。
- 如果 finetuning 时突然加 dropout, 会有 distribution shift。
- 所以 dropout 实验里, 作者先做 additional pretraining with dropout, 再 finetune。

结果:

- dropout 明显提升 finetuning baseline。
- dropout 明显改善 solution-level verifier, 缓解过拟合。
- token-level verifier 本来就更抗过拟合, 所以 dropout 提升较小, 但仍有一点 gain。

这提醒你: verifier 方法的成功不只是“加个 reward model”, 训练正则、数据大小、候选生成、多样性都影响最终效果。

## 13. Calculator annotations

GSM8K 解法里常见:

16 - 7 = <<16-7=9>>>9

训练时:

all tokens are normal LM tokens

测试时:

```
if model generates "<<expr="
    calculator evaluates expr
    system writes result tokens
    model continues generation after ">>"
```

论文用 Python eval 模拟 calculator。它也承认原版本 calculator 有一些小 bug，导致报告分数略低估；修复后 full GSM8K verification 大约提升 1% 左右，大多数实验影响小于 1%。

这和本仓库 tool\_aug\_math.py 有直接关系。真实生产中不能裸 eval 模型生成内容，必须 sandbox。本仓库 toy 版 \_safe\_exec\_block 会拒绝 import、open、os.、sys.、eval、exec 等危险模式。

## 14. 数学公式汇总

### Generator finetuning

x = question tokens  
y = solution tokens

```
L_lm(theta)
= - sum_t log p_theta(y_t | x, y_{<t})
```

### Sampling candidates

```
for i in 1..N:
    y_i ~ p_G(. | x, temperature=0.7)
```

### Labeling verifier data

```
ans_i = extract_final_answer(y_i)
z_i = 1 if ans_i == gold_answer else 0
```

### Token-level verifier loss

v<sub>i,t</sub> = verifier score after token t of solution y<sub>i</sub>

```
L_verifier(phi)
= mean_i mean_t (vi,t - zi)2
```

### Joint verifier objective

L<sub>joint</sub> = L<sub>lm</sub> + L<sub>verifier</sub>

### Test-time selection

```
chosen =
    argmax over i in 1..N:
        verifier_score(x, y_i)
```

## Top-ranked voting

```
top_K = K candidates with largest verifier scores
answer =
    most_common(extract_final_answer(y) for y in top_K)
```

## 15. 代码样例: sample-and-rank

下面是论文 verifier 推理逻辑的最小伪代码:

```
def sample_and_rank(question, generator, verifier, n=100):
    candidates = []
    for _ in range(n):
        solution = generator(question, temperature=0.7)
        score = verifier(question, solution)
        candidates.append((score, solution))

    candidates.sort(key=lambda x: x[0], reverse=True)
    return candidates[0][1]
```

如果 verifier 完美, 这个方法随  $n$  增大接近 test@N 上限。如果 verifier 不完美,  $n$  太大可能找到高分错误解。

## 16. 代码样例: final answer label

GSM8K 的 verifier 标签只看最终答案:

```
def label_candidate(solution, gold):
    pred = extract_gsm8k(solution)
    return int(numeric_equal(pred or "", gold))
```

这很便宜, 但它把“答案正确”和“推理正确”混在一起。后来 process supervision 要解决的正是这个问题: 不只看最终答案, 还看每一步是否正确。

## 17. 代码样例: top-K voting

```
from collections import Counter

def verifier_vote(question, generator, verifier, n=100, top_k=5):
    scored = []
    for _ in range(n):
        sol = generator(question, temperature=0.7)
        scored.append((verifier(question, sol), sol))

    top = sorted(scored, reverse=True)[:top_k]
    answers = [extract_gsm8k(sol) for _, sol in top]
    answers = [a for a in answers if a is not None]
    if not answers:
        return ""
    return Counter(answers).most_common(1)[0][0]
```

这段对应 Figure 7b 的思想。注意  $top\_k$  不是固定越大越好, 要和  $n$  一起调。

## 18. 和本仓库代码的对应关系

推荐按这个顺序读:

1. `learning/reasoning-eval/lectures/02-gsm8k.md` 先看 GSM8K 的题目格式、#### N、calculator annotation。
2. `learning/reasoning-eval/src/common.py` 看 `extract_gsm8k`、`numeric_equal`、`pass_at_k`。这对应 final answer extraction 和 `pass/test@k` 思想。
3. `learning/reasoning-eval/src/gsm8k_runner.py` 看 micro GSM8K 的 prompt、Let's think step by step、#### N 解析。
4. `learning/reasoning-eval/src/math_verify_demo.py` 这不是论文里的 verifier, 而是 evaluation verifier: 判断  $0.5$ 、 $1/2$ 、 $\frac{1}{2}$  是否等价。它帮助理解数学评测里“判答案”也是工程问题。
5. `learning/reasoning-eval/src/tool_aug_math.py` 对应论文 calculator annotation 的现代 tool-use 版本。模型写小代码, sandbox 执行。
6. `learning/reasoning-eval/src/capstone_reasoning_compare.py` 把 GSM8K/MATH/AIME/GPQA/Zebra 统一跑, 体会 reasoning evaluation matrix。

本地验证命令:

```
python learning/reasoning-eval/src/tests/test_reasoning.py
python learning/reasoning-eval/src/gsm8k_runner.py
python learning/reasoning-eval/src/math_verify_demo.py
python learning/reasoning-eval/src/tool_aug_math.py
```

30-60 分钟实验:

1. 打开 `gsm8k_runner.py`, 新增一个需要两步以上的题。
2. 让 dummy model 只输出 final answer, 不输出步骤。
3. 再写一个 mock model 输出步骤和 #### answer。
4. 比较 `extract_gsm8k` 是否稳定。
5. 手写 5 个候选解, 其中 2 个答案对、3 个答案错。
6. 写一个 toy verifier score 函数, 让它偏好包含中间计算的解。
7. 看 `sample-and-rank` 是否能挑到正确解。

## 19. 论文的局限和后来工作的方向

### Outcome label 不等于过程正确

verifier 标签是 `final answer correct`。这会让错误推理但答案碰巧正确的样本被标正。Appendix F 就有这种例子。

后来 process reward model 和 step verifier 的动机之一, 就是把监督从 outcome 移到 intermediate steps。

### Verifier 会被搜索攻击

Figure 7a 说明, 当 test-time samples 太多时, 性能会下降。这不是 generator 突然变差, 而是搜索找到了 verifier 高估的错误解。

这就是 reward hacking 的早期形式:

optimize against an imperfect judge  
-> find judge mistakes

## 数据集难度后来被追上

GSM8K 后来成为 chain-of-thought 和 reasoning model 的标准入口，也逐渐接近饱和。它今天仍重要，但更多是：

- 学习 math reasoning eval protocol。
- 学习 answer extraction。
- 学习 CoT prompt。
- 学习 verifier/reranking。
- 作为小规模 sanity check。

真正区分前沿模型时，社区会更多看 MATH、AIME、GPQA、HLE、ARC-AGI 等更难 benchmark。

## Calculator 和 tool-use 改变问题定义

论文里 calculator annotation 帮模型减少 arithmetic error。但一旦允许工具，benchmark 测的就不再是纯 LM 心算，而是 LM + tool protocol。

这不是坏事，但报告分数时必须说清楚：

no tool  
calculator annotation  
program-of-thought  
external solver

这些不能混报。

## 20. 对今天的意义

这篇论文连接了三个时代：

1. Dataset era: GSM8K 给数学文字题提供了高质量 benchmark。
2. Chain-of-thought era: 自然语言解法和 ##### answer 格式让模型可以显式展示中间步骤。
3. Test-time compute era: verifier sample-and-rank 说明推理能力不只来自参数和训练，也可以来自测试时多采样和选择。

今天你看到的很多方法都能在这里找到影子：

- self-consistency: 多采样后按最终答案投票。
- best-of-N: 多采样后用 reward/verifier 选。
- PRM: 对过程中的 step 打分。
- ORM: 对最终答案或完整 solution 打分。
- verifier-guided search: 用 judge 引导解空间搜索。
- tool-augmented reasoning: 外部 calculator/code 执行修复算术。

所以这篇不是过时 GSM8K 小论文。它是 reasoning evaluation 和 verifier-based inference 的基础读物。

## 21. AI agent 正确学习这篇论文的方法

第一轮: 不让 agent 总结, 让它考你 pipeline。

我正在读 Training Verifiers to Solve Math Word Problems。

请只问我问题, 不要给答案。

先考我 generator, verifier, test@N, sample-and-rank 的关系。

每次一个问题, 我答完后指出哪里不精确。

第二轮: 让 agent 查你的伪代码。

我会写 sample\_and\_rank 的伪代码。

请检查是否包含: N 个候选、temperature、verifier score、argmax selection、answer extraction、top-K voting 的区别。

第三轮: 用 agent 陪跑本地实验。

请陪我跑 reasoning-eval 的 test\_reasoning.py 和 gsm8k\_runner.py。

跑完后不要解释代码, 先让我说:

extract\_gsm8k 如何工作?

numeric\_equal 解决什么问题?

pass\_at\_k 和 test@N 有什么关系?

第四轮: 闭卷复述。

我用 300 字闭卷复述这篇论文。

请按 dataset、generator、verifier、experiments、limitations 五项各给 0-2 分。

只指出缺口, 不要替我重写。

## 22. 读完必须能闭卷回答

1. GSM8K 为什么要强调 high quality 和 high diversity?
2. GSM8K 的 train/test 数量是多少?
3. 为什么作者选择 grade school math, 而不是更难的 MATH?
4. 为什么自然语言解法比直接 final answer 更重要?
5. 6B direct final answer 从 20.6% 掉到 5.2% 说明什么?
6. test@1 和 test@100 分别衡量什么?
7. 为什么 generator 只 finetune 2 epoch?
8. verifier 的训练标签怎么来?
9. 为什么 final-answer correctness 会产生 false positive?
10. token-level verifier 和 solution-level verifier 有什么区别?
11. joint LM + verifier objective 为什么有帮助?
12. 为什么 large generator + small verifier 好于 small generator + large verifier?
13. 为什么 test-time samples 太多会让 verifier 方法退化?
14. top-ranked voting 和选择最高分 completion 有什么区别?
15. dropout 在这篇论文里解决了什么问题?
16. calculator annotation 在训练和测试时分别怎么用?
17. 这篇论文和后来的 PRM/best-of-N/self-consistency 有什么关系?
18. 在本仓库里你会打开哪些文件复现 GSM8K runner 和答案解析?

## 23. 一页复盘

Training Verifiers to Solve Math Word Problems 发布了 GSM8K, 并用它研究大模型的多步数学推理。GSM8K 有 8.5K 道高质量、多样化的小学数学文字题, 7.5K train 和 1K test, 每题通常需要 2

到 8 步，解法是自然语言过程加最终答案。论文发现只靠 finetuning generator 扩展不够理想，且完整自然语言解法非常重要：6B 模型若直接输出 final answer，性能从 20.6% 掉到 5.2%。作者提出 verification pipeline: generator 只训练 2 epoch 以保留 test@100 覆盖率；每个训练题采样 100 个 completion，用最终答案是否正确给候选解打标签；verifier 训练成 token-level value model，并保留 LM objective；测试时采样 100 个解，由 verifier 排序选择。实验显示 verification 在足够数据下显著优于 finetuning，6B verification 略超 finetuned 175B，约等于 30x model size boost；token-level verifier、joint objective、dropout 都能减少过拟合；test-time samples 增加有收益但过多会找到骗过 verifier 的错理解。这篇论文的现代意义是：推理模型不只是更大 generator，还包括候选生成、验证、投票、工具和测试时搜索。